

A READING PATH AND A TECHNICAL BRIEFING

Understanding Jev

The decision-only model class that returns typed answers and calibrated probabilities instead of text — what it is, why people are excited, when to reach for it, and what it is quietly displacing.

INSIDE

A sequenced reading list, from launch announcement to independent critique

The three primitives · the request and response, in full

Six proven uses · six documented failure modes

What it replaces, what it augments, and what it cannot touch

The vendor's numbers, and what independent testing found instead

Subject TypeSafe AI's Jev, released in limited early access 15 September 2026

Compiled September 2026, from the public record · every claim attributed

Caution Jev is weeks old. Treat performance figures as provisional and re-check them before they reach a design document.

0 How to use this document

Jev was announced on 15 September 2026. Almost everything written about it is younger than a month, much of it is by the vendor, and the independent work is only now arriving. This document is organized so that you can form your own view rather than inherit one.

IF YOU WANT	READ
The idea in five minutes	§1 and §2. That is enough to follow a conversation about it.
To decide whether to try it	§3 (when to use it), §4 (when not to), and §7 — the honest picture. §7 matters more than the marketing.
To explain it to your team	§5 (what it is replacing) and §6 (where it sits in an architecture).
To go deep	§9, the reading path — twelve pieces in the order that builds understanding fastest, with a note on what each one adds and how far to trust it.

ON THE STATE OF THE EVIDENCE

Three kinds of claim appear in this document and they are marked differently throughout. **Vendor claims** come from TypeSafe's own materials and benchmarks, which the company itself describes as designed by its own team. **Independent findings** come from third parties reproducing or testing those claims. Everything else is either directly observable from the API or is this document's own framing, and is flagged as such. Where the two disagree, both are given.

A NOTE ON WHAT THIS DOCUMENT IS NOT

It is not a tutorial — the official quickstart is better at that and is the first entry in the reading path. It is not a recommendation to adopt. A model four weeks old, behind a waitlist, from a company two years old, with vendor-designed benchmarks and one published independent calibration study, is a thing to *evaluate*, on a workload you own, against a baseline you already trust.

1 What Jev is

A model that does not write. You hand it some state and a set of bounded questions; it hands back typed answers, each with a probability. That is the entire surface area, and nearly everything interesting follows from the things it gave up to get there.

1.1 The one-paragraph version

Jev is a proprietary model from **TypeSafe AI**, a San Francisco company founded in 2024 by **Diogo Almeida** (CEO, previously four years at OpenAI on RLHF, InstructGPT, ChatGPT and GPT-4), **Erik Gafni** and **Sasha Sheng**. It entered limited early access on **15 September 2026**, alongside a **\$40 million seed round led by DCVC**. Unlike a large language model, it does not generate natural-language text. It accepts a state and a dictionary of questions, and returns typed values with probability estimates and confidence scores — output meant to be consumed by software, not read by a person. TypeSafe calls it the first “**System One model**”, a new model class rather than a variant of an existing one.

1.2 Why “System One”

The name borrows Daniel Kahneman's split between System 1 thinking — fast, automatic, intuitive — and System 2 — slow, effortful, deliberate. Frontier reasoning models are System 2: they think out loud, at length, expensively. A great deal of what production software actually asks of a model is not that. It is *is this relevant?*, *which bucket?*, *how urgent?*, *should this be blocked?* — questions with a small, known set of possible answers, asked millions of times.

TypeSafe's argument is that paying generation prices and generation latency for lookup-shaped work is a category error, and that a model built only for the lookup-shaped half can be faster and cheaper by orders of magnitude. The slogan version: **unstructured state in, typed probabilistic decisions out**.

The model's own name comes from **William Stanley Jevons**, the 19th-century economist whose paradox observes that making a resource cheaper tends to increase, not decrease, total consumption of it. Almeida has said the name reflects an expectation that far cheaper machine intelligence will be deployed far more widely — which is a thesis about the market, not a claim about the model.

1.3 The three primitives

Every question you can ask is one of three shapes. This is the whole vocabulary.

PRIMITIVE	ASKS	RETURNS
Choice	Pick one from a set of options you define. Up to 255 options.	The selection, a probability for every option, and a confidence score.
Score	Rate the state against an ordered rubric you define.	The score, a probability distribution across the levels, and a confidence score.
Noul	Evaluate a yes/no statement.	A single probability between 0 and 1.

THE PROPERTY THAT DOES THE WORK

All questions in a request are evaluated **independently and in parallel, in a single forward pass** against the same state. Adding a question barely changes the response time. This is the structural reason Jev is fast: it is not generating a sequence of tokens, so there is no sequence to wait for.

1.4 A request, and what comes back

The API is one endpoint, `POST https://api.typesafe.ai/v1/systemone`, with one documented model route, `jev-latest`. Official SDKs exist for Python and JavaScript; the community has since written many more (§9).

```
# Python – the shape of every Jev call
from typesafe_sdk import Noul, TypeSafeClient

jev = TypeSafeClient()

r = jev.system_one(
    state={"query": query, "page": page_markdown[:8000]},
    questions={
        "answers_query": Noul(
            instructions="Does this page directly answer the query?",
            criteria={
                "true": "Explains the specific behavior queried",
                "false": "Related topic but doesn't answer",
            },
        ),
    },
)

r.answers["answers_query"].noul  # → 0.0 ... 1.0, a calibrated probability
```

Note what is absent: no prompt template, no output parser, no retry-on-malformed-JSON, no “respond only with valid JSON and nothing else”. The answer arrives typed because the type is part of the request.

1.5 How it differs from a language model

	FRONTIER LLM	JEV
Output	Free-form text, token by token	Typed values from a schema you supplied, plus probabilities
Generation	Sequential — each token waits for the last	One parallel pass over all questions
Latency	Seconds to minutes for reasoning tasks	70–500 ms end to end
Hallucination	Can invent anything, including malformed output	Cannot produce a value outside the schema. It can still choose the wrong valid option
Confidence	Not natively available; must be inferred or elicited	A probability per option, by construction
Training	Human preference (RLHF) among others	Synthetic data only, via Reinforcement Learning for Calibrated Decisions (RLCD) — optimizing probabilities against outcomes rather than preferences
Price	Dollars per million tokens	\$0.042 per million input tokens; output free, “too cheap to meter”
Good at	Writing, open-ended reasoning, multi-hop, arithmetic	Bounded decisions over a known answer space, at volume

WHAT TYPESAFE HAS NOT PUBLISHED

There is no technical paper, no architecture description, and no weights. Observers have suggested it may be built on open-weight LLMs. The company describes the model as transformer-based and trained exclusively on synthetic data, and that is roughly the limit of what is public. Judge it as a black box with a measurable interface, because that is what it currently is.

2 Why it matters

Three arguments are being made for Jev. One is economic, one is structural, and one is about what you can build when a decision costs a fraction of a cent. They are not equally strong.

2.1 The economic argument

The headline numbers are striking and they are the vendor's own. TypeSafe publishes workflow evaluations at `evals.typesafe.ai`:

ON TYPESAFE'S DECISION TASKS	JEV	GPT-6 SOL
Accuracy	76.0%	79.1%
Cost per call	\$0.0001	\$0.2152
Latency	0.4 s	34.3 s

From figures like these TypeSafe derives its headline multipliers: **193.6× faster and 444.6× cheaper** at the extreme. The company is unusually candid about the caveat, noting the benchmarks were “designed by TypeSafe's own capabilities team” and that such multipliers represent “the higher end of real world gains”.

READ §7 BEFORE YOU QUOTE ANY OF THESE NUMBERS

A TypeSafe employee measured a **15.9% speed improvement** on a real pipeline when substituting one step with Jev, with cost down 30.1%. Component speed-ups do not become pipeline speed-ups when the other calls in the pipeline are unchanged. The number that decides an adoption is the pipeline number, and it is not the one in the headline.

2.2 The structural argument

This one is more durable than the benchmarks, because it follows from the design rather than from a measurement.

- **The output cannot be malformed.** Values outside the supplied schema are not representable, so the entire category of parse-retry-reprompt logic that surrounds LLM classification simply is not needed. TypeSafe quotes a **0% structured-output error rate**; structurally, it is hard to see how it could be otherwise.
- **Confidence arrives with the answer.** Not a self-reported “I am 90% sure” in prose, which is famously unreliable, but a probability per option. That makes *abstention* a first-class option: route the confident 95% automatically and send the rest to a human or a bigger model.

- **Latency is bounded and roughly flat in the number of questions.** A single parallel pass means you can ask twelve questions about a document for approximately the price and time of asking one.

THE HONEST FRAMING OF “CANNOT HALLUCINATE”

It cannot hallucinate *outside your schema*. It can absolutely pick the wrong option inside it. “Structural correctness is not semantic correctness” is the most useful sentence written about this model so far, and it belongs in any summary you give a colleague who has only heard the marketing.

2.3 The Jevons argument

The third argument is the one the model is named after. At roughly **\$0.0004 per decision** against \$0.03–\$0.18 for a frontier model, some things stop being projects and become defaults:

- Screening every fetched page for prompt injection, rather than the suspicious ones.
- Judging every tool call a coding agent proposes, rather than sampling them.
- Classifying an entire documentation site — TypeSafe demonstrates 500+ pages for **under ten cents**.
- Reranking search results inside a request, where three seconds of LLM latency was previously prohibitive.

Whether that is transformative or merely convenient depends entirely on whether the decisions are good enough at that price, which is the question §7 is about. But the direction is real: TypeSafe notes the pricing “may be subsidized” and that it expects prices “to go down, not up”.

2.4 What practitioners are reporting

SOURCE	REPORTED
Vercel engineer Pranit Sharma	5–18× faster results with greater accuracy than OpenAI's Luna 5.6 on his workload.
An unnamed developer, via TechCrunch	10–20× less expensive than Gemini , with confidence scores usable for workflow automation.
Every's evaluation	580× cheaper, but slightly less accurate — 6 of 7 planted defects caught, against 7 of 7 for Fable 5.1.
Independent field measurements	1.16× to 6× faster across scenarios; the 6× figure came from a production tax-document pipeline.
The <code>pi-warden</code> project	Over 17,000 recorded tool calls, Jev held 42, and about 88% of those holds were correct.

Note the spread: 1.16× to 580× depending on what is being measured and against what. That spread is the single most important fact in this section.

3 When to use it

The test is simple: can you write down every acceptable answer before you ask the question? If yes, Jev is a candidate. If no, it is not.

3.1 Six uses that have been demonstrated

These are not hypotheticals — each has a published implementation or measurement behind it.

USE	WHAT IT LOOKS LIKE, AND WHAT WAS MEASURED
Rerank retrieval results	Score each candidate page for whether it actually answers the query, before any of it reaches an agent's context. On TypeSafe's legal retrieval benchmark this moved top-1 accuracy from 5% to 18% after an initial BM25 ranking, at negligible cost.
Verify agent citations	Ask whether a quoted passage supports, contradicts, or simply ignores the claim made from it. A published test verified four accurate citations and caught four planted failures at 0.99 confidence .
Judge a coding agent's tool calls	Before executing a command, ask whether it is irreversible, off-task, out of scope, or mismatched with stated intent. <code>pi-warden</code> reports 42 holds over 17,000 calls, ~88% of them correct .
Screen fetched pages for prompt injection	Four yes/no questions gating whatever enters an agent's context: relevance, evidence quality, premise contradiction, injection attempt. Caught an injected instruction while preserving legitimate corrective information.
Route between models	Estimate task difficulty and whether web access is needed, then send easy work to a cheap model. Vercel's AI SDK 7 exposes this directly through an <code>evaluate</code> method.
Classify at scale	Label an entire crawled site — page type, deprecation status, presence of code examples. 500+ pages for under ten cents at list price.

3.2 The pattern underneath them

Every one of those is a **filter, a gate, a router, or a judge** — a decision *about* content, made at high volume, where the answer space is small and known in advance, and where being slightly wrong occasionally is survivable because the decision is cheap enough to check or repeat.

Notice what none of them is: the thing that produces the final answer for a user. Jev sits in the layer *around* the model that does that.

3.3 A decision table

YOUR SITUATION	REACH FOR
Bounded answer space, high volume, latency matters	Jev
You need a confidence number to decide whether to escalate	Jev — this is its distinguishing feature
You are currently prompting an LLM for JSON and parsing it	Jev , if the JSON is a classification rather than content
The rule is genuinely fixed and expressible in code	Code . A regex is faster, free, and auditable
You need prose, a summary, an explanation, or an argument	An LLM . Jev is not trained to produce text
Multi-hop reasoning, arithmetic, or date maths	An LLM , or code — see §4
Low volume, and a single error is expensive	An LLM , or a human. The economics that justify Jev are not present
You cannot enumerate the acceptable answers	Not Jev . This is the disqualifying condition

4 When not to use it

TypeSafe publishes a document it calls “jaggedness” — an unusually direct list of the things its own model is bad at. It is the most useful page the company has written, and it is summarized here because these failure modes are not obvious from the API.

DO NOT USE IT FOR	WHY, AND WHAT TO DO INSTEAD
Maths and counting	Jev recognizes the <i>shape</i> of an answer rather than tallying. Count in code and pass the count in as state.
Date comparison	Dates are treated as text, not as ordered values. Extract and compare them programmatically, then ask Jev about the result.
Indirection	Multi-hop questions — “is the author of the linked document the same person who filed the ticket?” — lose accuracy. Reduce the hops, or name the relevant state fields explicitly.
Large, noisy state	Unrelated detail distracts it. Filter before sending; do not paste the whole document and hope.
Text generation	It is not trained to produce free-form text at all. This is not a weakness, it is the design.
Adversarial input	Injected instructions can shift answers. Write precise criteria and test edge cases — and note the irony that one of its best uses is <i>detecting</i> injection.

4.1 The failure mode nobody expects

WITH JEV, THE QUESTION TEXT IS THE PROGRAM

Independent testing found that **wrong criteria descriptions produced 16.7% accuracy — below the 25% random floor** on a four-option choice. A model that does worse than chance is not being unlucky; it is being precisely misled. Your `instructions` and `criteria` strings are not documentation or flavor text. They are the logic, and they deserve the review, the tests and the version control you would give any other logic.

4.2 One big question, or several small ones

A finding worth internalizing before you design anything. On a reproducible 2,000-email phishing benchmark, Jev scored **62.6% when asked a single question** — and **95.0% when the same judgment was split into five narrow questions**, with weights fitted on 1,000 labeled examples.

Two things follow. First, decomposition is not a micro-optimization here; it is the difference between unusable and good. Second, that second number required a labeled training set and fitted weights — which is to say it was an *engineering project*, not a prompt. Budget accordingly, and be suspicious of any quoted accuracy that

does not say which of the two regimes it came from.

PRACTICAL CONSEQUENCE

Because all questions run in one parallel pass at roughly flat cost, asking five narrow questions instead of one broad one is nearly free. The architecture actively rewards the thing that also happens to improve accuracy. Design for many small questions from the start.

5 What it is replacing

The displacement is economic and operational rather than capability-based. Nothing here was impossible before; it was priced and timed such that you did it sparingly, or not in the request path, or not at all.

BEING DISPLACED	THE OLD ARRANGEMENT, AND WHAT CHANGES
LLM classifiers	The dominant pattern for three years: prompt a frontier model, demand JSON, parse it, retry when it comes back malformed. Jev removes the prompt engineering, the parser, and the retry loop, and runs the same work 40–200× faster at a small fraction of the cost. This is the biggest single category.
Guardrail and safety models	A second expensive model watching the first one for jailbreaks, policy violations and off-task behavior. That monitoring was often sampled rather than universal because of cost. At Jev prices it can run on every call.
Rerankers	Cross-encoder rerankers do relevance well but take a fixed notion of relevance. Jev accepts free-form instructions and can judge semantic relationships beyond "is this similar" — <i>does this actually answer the question, does this contradict the premise</i> — at latencies that fit inside a request.
Rules engines and hand-written heuristics	Regexes, decision trees and keyword lists are fast and free but understand no meaning, and they rot. Jev reads context. The trade is that you give up the exact auditability of a rule.
Zero-shot and fine-tuned small classifiers	A trained classifier per task means a training set, a pipeline, and a retraining cadence per task. Jev is zero-shot against instructions you can edit in place.
Doing nothing	The quiet one. Plenty of checks were never implemented because a model call per item could not be justified. Screening every page, judging every tool call, labeling every document — these were not rejected on principle but on budget.

5.1 What it is not replacing

IT IS NOT AN LLM REPLACEMENT, AND NOBODY SERIOUS IS CLAIMING IT IS

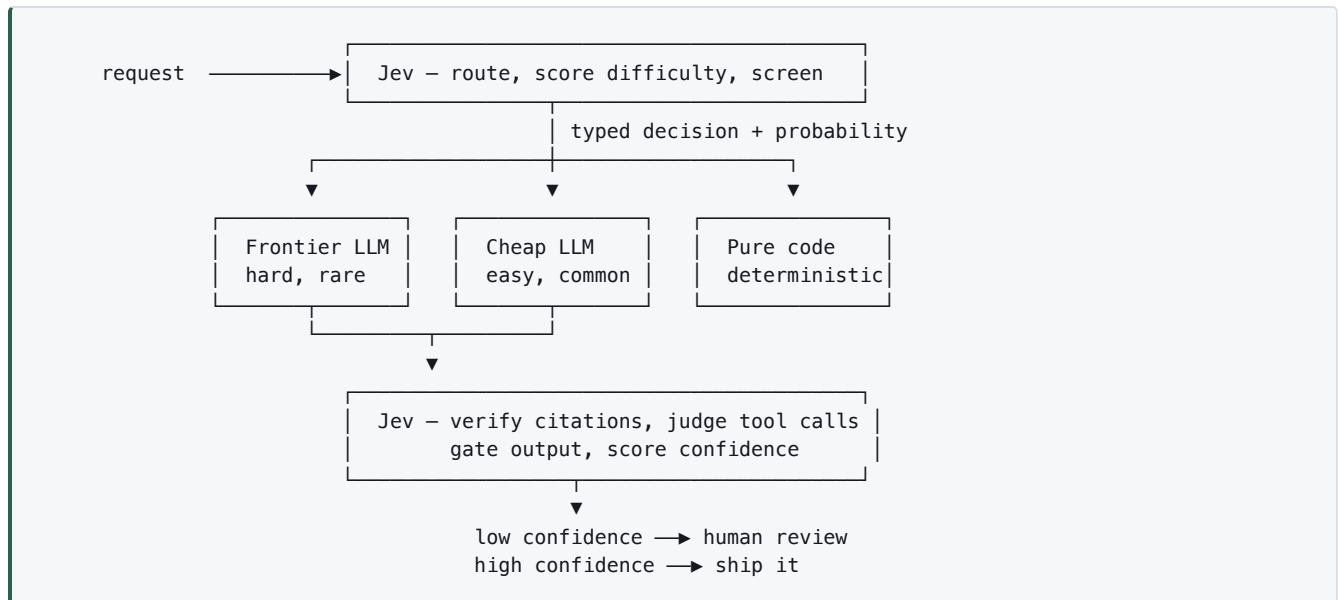
The consistent framing from TypeSafe, from practitioners, and from independent reviewers is that Jev works **in concert with** language models. It occupies the decision layer — filtering, routing, judging, verifying — while generation, open-ended reasoning and anything requiring explanation stay where they are.

- **Chat and assistants** — untouched. There is no text.
- **Agents** — untouched as planners. Jev is increasingly the thing *around* the agent: its router, its guard, its context filter.
- **Code generation** — untouched, though Jev is being used to judge what generated code is allowed to do.

- **Embeddings and vector search** — complementary. Retrieve with embeddings, then rerank with Jev.
- **Deterministic code** — if a rule is genuinely fixed, code remains faster, free, and auditable. Do not put a model where an `if` belongs.

6 Where it sits

The architectural pattern that has emerged in the first weeks is task-specific routing: not one model per company, but a mix of model types chosen per task.



Jev appears twice, and the second appearance is the more interesting one. Because the model returns a probability rather than an assertion, the confidence score becomes a routing primitive: automate the confident majority, escalate the tail. That is a shape you could not easily build when the only available signal was text that claimed to be confident.

7 The honest picture

Everything in this section is independent of TypeSafe. It is the part that a design document should cite, and it is the part least represented in the coverage so far.

7.1 The benchmark claims do not survive contact with a pipeline

THE CLAIM	WHAT CLOSER READING SHOWS
193.6× faster	A single decision call against a frontier model, on workflows TypeSafe wrote. A TypeSafe employee measured 15.9% faster on a real pipeline when swapping one step to Jev — cost fell 30.1%, but the other calls in the pipeline still dominated. Component gains do not scale to system gains.
444.6× cheaper	Same basis. Independent field measurements range from 1.16× to 6× , the 6× coming from a production tax-document pipeline.
76.0% accuracy	The reference answers are averages of GPT-6 Astra and Fable 5.1 — which creates a ceiling where Jev cannot outperform the models used as the grading key . This is agreement with a model-derived reference, not independently verified correctness.

7.2 The calibration question

Calibration is the whole proposition. If the probability is honest, you can route on it; if it is not, the confidence score is decoration. An independent study tested Jev on 900 rule-generated support tickets it could not have seen, plus three public benchmarks, for about \$0.06 in API cost.

WHAT IT FOUND

Measured calibration error was 4.4× the noise floor. The chosen level carried 0.74 probability on average, and a temperature of 3.4 was required to make the probabilities honest. In plain terms: on that out-of-distribution task, Jev was **overconfident**, and the raw numbers needed rescaling before they could be trusted as probabilities.

TypeSafe has not published standard calibration metrics across independent ground-truth tasks. If you intend to route on confidence — and that is the main reason to choose this model — **measure calibration on your own data and fit your own temperature before you trust a threshold**. The study is reproducible for pennies; there is no excuse for skipping it.

7.3 Is there a moat?

An unusually large open-source response appeared within days: dozens of “open System One” projects reproducing the interface on open-weight models — Qwen, Gemma, ModernBERT, llama.cpp, MLX.

The most direct comparison so far: a community project reproduced Jev's interface and reached **0.845 modal agreement against Jev's published 0.883**. The interpretation offered, and it seems right, is that **the claimed moat is the training data for calibration, not the architecture**. If that holds, the interface commoditizes quickly and the differentiator is how honest the probabilities are — which is exactly the thing §7.2 says is not yet independently established.

7.4 Adoption risks worth writing down

- **Age.** Four weeks old, limited early access, waitlist-gated, one documented model route (`jev-latest`) with no versioning story yet visible.
- **Single vendor, no weights.** No paper, no architecture, no self-hosting. Your decision layer becomes an API dependency on a two-year-old company.
- **Subsidized pricing.** TypeSafe says the price may be subsidized. Model the economics at a price that is not.
- **A moving `latest`.** If the route floats and your criteria strings are the program, a silent model update is a silent logic change. Pin what you can, and keep a regression set.
- **Criteria are load-bearing and fragile.** §4.1 — below-random accuracy from bad criteria strings is a failure mode with no analog in a rules engine.

A REASONABLE POSITION

The structural argument (§2.2) is sound and does not depend on any contested number: typed output, parallel evaluation, and a probability per option are real properties of the design. The performance claims are unproven at pipeline level, and the calibration claim — the one that matters most — has one independent study and it was not flattering. Pilot it on a real workload, measure calibration yourself, and keep the fallback path warm.

8 Getting started

Access	Waitlist, then an API key from the TypeSafe dashboard.
Endpoint	POST <code>https://api.typesafe.ai/v1/systemone</code>
Model route	<code>jev-latest</code> — the only one documented
Official SDKs	<code>pip install typesafe-sdk</code> · JavaScript/TypeScript
Without a key	Reachable through Vercel AI Gateway, Cloudflare Workers AI, Netlify AI Gateway and OpenRouter
Price	\$0.042 / M input tokens; output free
Limits	<code>Choice</code> supports up to 255 options; higher cardinality uses a two-stage approach

A SENSIBLE FIRST PROJECT

Take one classification you already run through an LLM. Keep the old path. Run both for a week on live traffic, logging Jev's answer, its probability, and the incumbent's answer. You will get three things at once: an accuracy comparison on *your* data, a calibration curve you can fit a threshold to, and a cost figure that is a pipeline figure rather than a component one.

9 The reading path

Twelve pieces, in the order that builds understanding fastest. Each entry says what it adds that the previous one did not, and how far to trust it. Stages 1 and 2 are about two hours; stage 3 is the one that will change how you design.

9.1 Stage one — orientation

01 Introducing System One Models and Jev — TypeSafe

typesafe.ai/blog/introducing-system-one-models-and-jev

Start here. The primary source: the claim in the company's own words, the System One framing, and the headline numbers. Read it first so you recognize later which parts of the discourse are the vendor's framing being repeated. Vendor. Take the argument, not the multipliers.

02 A new kind of AI model from a ChatGPT inventor is thrilling developers — TechCrunch

techcrunch.com/2026/09/18/a-new-kind-of-ai-model-from-a-chatgpt-inventor-is-thrilling-developers/

The outside view, and the first practitioner numbers — the Vercel engineer reporting 5–18×, another developer reporting 10–20× cheaper than Gemini. Also the clearest statement of Almeida's motivation: “computers speak a different language”. Press. Light on skepticism.

03 TypeSafe AI's Jev: What “System One Models” Actually Are — TrueFoundry

truefoundry.com/blog/typesafe-ai-jev

The best explanation of the *category* rather than the product: the Kahneman framing, the three properties traded for giving up text, and the task-specific routing architecture that §6 of this document is drawn from. Independent vendor blog. Conceptually strong.

9.2 Stage two — how it actually works

04 What Is Jev? Inside TypeSafe's Decision-Only AI Model — Firecrawl

firecrawl.dev/blog/what-is-jev

The single most useful third-party piece. Real code, the API shape, six worked use cases with measurements, and honest comparisons against LLMs, classical classifiers and embedding rerankers. If you read only one thing after the launch post, read this. Independent, technical, balanced.

05 Quickstart, primitives and cookbooks — TypeSafe docs

docs.typesafe.ai/introduction/quickstart

Where to actually learn the three primitives properly and see the patterns the vendor recommends. Pair it with the API reference at docs.typesafe.ai/api when you start writing calls. Vendor. Reference material; reliable for mechanics.

06 Jev: TypeSafe's System One Model Explained — DataCamp

datacamp.com/blog/system-one-models-jev

The gentlest treatment of the primitives with worked examples, and the clearest list of what Jev displaces — LLM classifiers, guardrail models, rules engines, rerankers. Useful if you need to explain this to someone who does not write code. Educational. Repeats vendor numbers uncritically.

07 The jaggedness documentation — TypeSafe docs

docs.typesafe.ai — the limitations page

The six documented failure modes summarized in §4: maths, dates, indirection, noisy state, generation, adversarial input. A vendor publishing this is unusual and it should raise your trust in the rest of their documentation. Vendor, self-critical. Read before designing anything.

9.3 Stage three — the limits

THIS IS THE STAGE MOST READERS SKIP

Stages 1 and 2 will leave you enthusiastic. Stage 3 is what turns enthusiasm into a design that survives contact with production. Do not stop before it.

08 Independent calibration test on out-of-distribution tasks — [scienthoon/jev-ood-calibration](https://github.com/scienthoon/jev-ood-calibration)

github.com/scienthoon/jev-ood-calibration

The most important item on this list. 900 rule-generated support tickets Jev cannot have seen, plus three public benchmarks, with raw responses, expected calibration error against a noise floor, and a temperature refit. Found calibration error $4.4\times$ the noise floor and a required temperature of 3.4. **Reproducible for about \$0.06** — so reproduce it, on your data. Independent, reproducible, adversarial in the good sense.

09 TypeSafe Claims Its Model Is 193.6× Faster. Its Own Employee Measured 15.9% — Cherry Creek News

thecherrycreeknews.com/jev-typesafe-benchmark-checked-explainer-wave-cherry_creek/

Why component benchmarks mislead: swapping one pipeline step gave 15.9% speed and 30.1% cost, not 193.6×. Also spots that the reference answers are averages of the very models Jev is compared against, creating a ceiling it cannot exceed. Independent, skeptical. The counterweight to item 01.

10 62.6% Asked Once and 95% Split Five Ways — The D*AI*LY BRIEF

beri.net/article/typesafe-jev-typed-decision-model-calibration-decomposition-shadow-eval

The decomposition finding, on 2,000 phishing emails, and the below-random 16.7% result from wrong criteria strings. Between them these two numbers are the strongest practical design guidance available: **ask many narrow questions, and treat criteria as code**. Independent, quantitative.

9.4 Stage four — the ecosystem

11 The Ultimate Jev Resource List 2026 — ScriptByAI

scriptbyai.com/jev-resource-list/

A remarkable map of what appeared in weeks: official and community SDKs in a dozen languages, gateways, dozens of MCP servers and agent skills, browser and computer-use agents, benchmarks, and a long list of open-weight “System One” reimplementations. Skim it for the shape of the ecosystem, then mine it for the three projects nearest your problem.

Link directory. Unvetted; treat as an index.

12 Official workflow evaluations — `evals.typesafe.ai`

evals.typesafe.ai

Read this *after* items 08 to 10, not before. With the caveats in hand you can see what the evaluation is actually measuring — agreement with a model-derived reference on vendor-designed workflows — and read the numbers for what they are worth, which is not nothing. Vendor. Now readable critically.

IF YOU ONLY HAVE AN HOUR

Items **04** (Firecrawl, how it works), **08** (the calibration study) and **09** (the benchmark critique). That combination gives you the mechanism, the strongest independent evidence, and the correct level of skepticism — and it is enough to hold your own in an architecture review.

10 Glossary and sources

10.1 Terms used in this document

TERM	MEANING HERE
System One model	TypeSafe's proposed model class: unstructured state in, typed probabilistic decisions out, no text generation. Named for Kahneman's fast, automatic System 1 thinking.
Choice / Score / Noul	The three question primitives. Pick one of N; rate against a rubric; yes/no probability.
State	The context you pass in — a string, JSON, or array of text — that all questions are evaluated against.
Calibration	Whether a stated probability matches observed frequency. A model is calibrated if things it calls 70% likely happen about 70% of the time. The property Jev's value rests on.
Expected calibration error (ECE)	The standard measure of that gap. Meaningful only against a noise floor, which is why item 08 in the reading path computes one.
Temperature refit	Rescaling a model's probabilities after the fact so they become honest. A temperature of 3.4 means the raw outputs were substantially overconfident.
Decomposition	Splitting one broad question into several narrow ones. On the phishing benchmark this moved accuracy from 62.6% to 95.0%.
RLCD	Reinforcement Learning for Calibrated Decisions — TypeSafe's stated training approach, optimizing probabilities against outcomes rather than human preferences.
Jaggedness	TypeSafe's own term for the uneven capability surface: strong on bounded judgment, weak on maths, dates and indirection.

10.2 Sources

Everything in this document is drawn from the following, consulted September 2026. Where a figure is attributed to TypeSafe it is marked as a vendor claim in the text; where it comes from a third party it is marked as an independent finding.

SOURCE	USED FOR
Wikipedia, <i>Jev (AI model)</i> en.wikipedia.org/wiki/Jev_(AI_model)	Founders, funding, timeline, architecture summary, RLCD, the Jevons naming, the vendor's own caveat on its multipliers.
TechCrunch, 18 Sept 2026 techcrunch.com/2026/09/18/...	Practitioner reports (Vercel, Gemini comparison), Almeida's framing, the agent-monitoring use.
Firecrawl, <i>What Is Jev?</i> firecrawl.dev/blog/what-is-jev	API shape, code example, the six use cases and their measurements, comparisons against classifiers and rerankers, the jaggedness list, the openjev agreement figure.

SOURCE	USED FOR
TrueFoundry, <i>What “System One Models” Actually Are</i> truefoundry.com/blog/typesafe-ai-jev	The category framing, the Kahneman derivation, the task-specific routing architecture.
DataCamp, <i>System One Models: Jev</i> datacamp.com/blog/system-one-models-jev	Primitive examples, per-decision pricing, the displacement list.
Developers Digest, release guide developersdigest.tech/blog/typesafe-jev-system-one-models-release-guide-2026	Benchmark table against GPT-6 Sol, pricing, endpoint, SDKs, the 255-option limit.
Cherry Creek News, benchmark analysis thecherrycreeknews.com/jev-typesafe-benchmark-checked-explainer-wave-cherry_creek/	The 15.9% pipeline measurement, the 1.16–6× field range, the grading-key ceiling.
<i>jev-ood-calibration</i> github.com/scienthoon/jev-ood-calibration	The independent calibration study: 4.4× noise floor, 0.74 mean probability, temperature 3.4.
The D*AI*LY BRIEF beri.net/article/typesafe-jev-typed-decision-model-calibration-decomposition-shadow-eval	62.6% → 95.0% decomposition on 2,000 phishing emails; the 16.7% wrong-criteria result.
ScriptByAI, resource list scriptbyai.com/jev-resource-list/	The ecosystem map: SDKs, gateways, MCP servers, open reimplementations, benchmarks.
TypeSafe official typesafe.ai · docs.typesafe.ai · evals.typesafe.ai	Launch post, quickstart, API reference, jaggedness documentation, workflow evaluations.

SHELF LIFE

This document describes a model four weeks old. Pricing, limits, the model route, the availability of weights, and above all the independent evidence base will all change. The reading path in §9 is the durable part — those sources will keep being updated. The numbers in §2 and §7 should be re-checked before they are quoted anywhere that matters.

Understanding Jev · compiled September 2026 from the public record. Vendor claims and independent findings are distinguished throughout; where they conflict, both are given and neither is resolved for you.